Estadística Computacional
2do Cuatrimestre 2026
Trabajo Práctico

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Análisis estadístico de conteos de operaciones

Objetivo

El objetivo de este trabajo es estudiar estadísticamente el *costo* de algoritmos de ordenamiento, medido como la cantidad de operaciones elementales (comparaciones e intercambios) que realizan, y analizar cómo cambia en función del tamaño y del grado de desorden de la entrada.

Introducción

Una forma habitual de medir el costo de un algoritmo es cronometrar su ejecución. Sin embargo, el tiempo medido mezcla el costo del algoritmo con ruido ajeno a él (el sistema operativo, la memoria caché, otros procesos, la resolución del reloj), y ese ruido cambia de una computadora a otra y de una corrida a otra.

En este trabajo vamos a medir el costo *contando operaciones*: cada vez que el algoritmo compara dos elementos de la lista sumamos uno a un contador de *comparaciones*, y cada vez que intercambia dos posiciones sumamos uno a un contador de *swaps*. Estos conteos no dependen de la máquina y son exactamente reproducibles.

Pero entonces, ¿de dónde sale la aleatoriedad? Los algoritmos que vamos a usar son **deterministas**: con la misma lista de entrada hacen siempre exactamente las mismas operaciones. El azar entra **por la entrada**: si la lista a ordenar es elegida al azar, la cantidad de operaciones es una **variable aleatoria**. Una ventaja importante respecto de los tiempos es que para varios de estos conteos **la distribución verdadera es conocida**, de modo que vamos a poder contrastar nuestras estimaciones, intervalos y tests contra el valor real.

Algoritmos

Vamos a trabajar principalmente con *insertion sort* y con *quicksort*.

Algoritmo 1 Insertion sort instrumentado

```
1: función InsertionSort(a[0..n − 1])
2:     swaps ← 0;  comp ← 0
3:     para i ← 1 hasta n − 1 hacer
4:         j ← i
5:         mientras j > 0 hacer
6:             comp ← comp + 1
7:             si a[j − 1] > a[j] entonces
8:                 intercambiar a[j − 1] y a[j];  swaps ← swaps + 1;  j ← j − 1
9:             si no
10:                salir del mientras
11:    devolver (swaps, comp)
```

Algoritmo 2 Quicksort instrumentado (pivote = primer elemento, partición de Lomuto)

```
1: función Quicksort(a[lo..hi])
2:     si lo ≥ hi entonces devolver
3:     p ← a[lo];  i ← lo
4:     para j ← lo + 1 hasta hi hacer
5:         comp ← comp + 1
6:         si a[j] < p entonces
7:             i ← i + 1;  intercambiar a[i] y a[j];  swaps ← swaps + 1
8:     intercambiar a[lo] y a[i];  swaps ← swaps + 1
9:     Quicksort(a[lo..i − 1]);  Quicksort(a[i + 1..hi])
```

El código base incluye además *selection sort* y una variante de quicksort cuyo pivote es el elemento del medio del subarreglo.

Entradas aleatorias

Las listas de entrada son permutaciones de $0, 1, \ldots, n-1$ (sin repetidos). Usaremos dos modelos de entrada:

- **Permutación uniforme:** las $n!$ permutaciones tienen la misma probabilidad.
- **Desorden controlado k:** se parte de la lista ordenada y se aplican k *transposiciones al azar* (en cada paso se eligen dos posiciones i, j uniformes e independientes y se intercambian). Con $k = 0$ la lista está ordenada; a medida que k crece la lista se parece cada vez más a una permutación uniforme.

Metodología

Entregamos un código base en Python (`codigo_base.py`) que implementa los algoritmos con sus contadores, los dos generadores de entradas y una función `medir` que corre varios algoritmos **sobre las mismas listas**. El archivo `codigo_base.py` es la única definición válida de los contadores: no modifiquen dónde se cuenta una comparación o un swap, porque cambiarían las distribuciones.

Al ejecutarlo (tarda alrededor de medio minuto) se obtiene un archivo `conteos.jsonl` con una medición por renglón:

```
{"experimento": "uniforme", "algoritmo": "insertion", "n": 100, "k": null,
 "lista": 0, "swaps": 2311, "comparaciones": 2406}
```

donde `experimento` indica a qué corrida pertenece la medición (`uniforme`: $N = 2000$ permutaciones con $n = 100$; `tamanos`: $N = 200$ permutaciones para cada n; `desorden`: $N = 300$ listas con $n = 100$ para cada k), `k` es `null` para permutaciones uniformes y `lista` identifica la entrada (todas las filas con el mismo `experimento`, `n`, `k` y `lista` corresponden a la misma lista). Cada grupo debe fijar su propia semilla.

Notación

Para una permutación uniforme de tamaño n llamaremos S_n a los swaps de insertion sort, C_n a sus comparaciones y Q_n a las comparaciones de quicksort con pivote en el primer elemento. $H_n = \sum_{j=1}^{n} 1/j$ es el n-ésimo número armónico.

Los parámetros se escriben con letras griegas y sus estimadores con la misma letra y un sombrero (o una tilde, si hay más de un estimador del mismo parámetro). Para $n = 100$ escribimos $\mu_S = \mathrm{E}(S_{100})$, $\sigma_S^2 = \mathrm{Var}(S_{100})$, y análogamente μ_Q, σ_Q^2. Por ejemplo, a partir de N listas, $\hat{\mu}_Q$ es la media muestral de las comparaciones de quicksort.

En todo el trabajo n es el tamaño de cada lista, N la cantidad de listas de una muestra, R la cantidad de repeticiones de un experimento de simulación y k el desorden de la entrada.

Coeficiente de asimetría. Para una variable aleatoria X con media μ y desvío σ, el coeficiente de asimetría es

$$\gamma_X = \frac{\mathrm{E}\left[(X-\mu)^3\right]}{\sigma^3}.$$

A partir de una muestra $x_1, \ldots, x_N$ con media $\bar{x}$ se estima con

$$\hat{\gamma}_X = \frac{\frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^3}{\left(\frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^2\right)^{3/2}},$$

que es lo que calcula `scipy.stats.skew` con sus opciones por defecto.

Resultados conocidos

Para quicksort con pivote en el primer elemento sobre una permutación uniforme se sabe (Knuth, *The Art of Computer Programming*, vol. 3) que

$$\mathrm{E}(Q_n) = 2(n+1)H_n - 4n \approx 2n\ln n, \qquad \mathrm{Var}(Q_n) = 7n^2 - 4(n+1)^2 H_n^{(2)} - 2(n+1)H_n + 13n,$$

donde $H_n^{(2)} = \sum_{j=1}^{n} 1/j^2$. Estas fórmulas se usan como dato; no es necesario demostrarlas.

Test estadísticos

Nos interesa usar los conteos para contrastar hipótesis como:

- La distribución de los conteos es normal.
- Dos algoritmos (o dos modelos de entrada) tienen el mismo costo medio.
- El exponente de crecimiento del costo medio con n es el que predice la teoría.

Regresión lineal

Para estudiar la *complejidad* usaremos regresión lineal simple sobre variables transformadas. Si el costo medio crece como una potencia, $\mathrm{E}(S_n) \approx A\, n^b$, tomando logaritmos se obtiene el modelo lineal

$$\ln S_n = \beta_0 + \beta_1 \ln n + \epsilon,$$

donde β_1 estima el exponente b. Para quicksort, en cambio, $\mathrm{E}(Q_n)/n$ crece linealmente con $\ln n$, y el modelo propuesto es

$$\frac{Q_n}{n} = \beta_0 + \beta_1 \ln n + \epsilon.$$

Tareas

Bloque 1: Estadística descriptiva y modelo

1.1. **Entradas y conteos.**

a) Una permutación uniforme de tamaño n es una lista elegida al azar entre las $n!$ permutaciones de $0, \ldots, n-1$, todas con la misma probabilidad. Con la función del código base que genera permutaciones uniformes, genere 6000 permutaciones de tamaño $n = 3$ y arme una tabla con la frecuencia de cada una de las 6 posibles. ¿Qué distribución tiene la frecuencia de una permutación fija? ¿De qué tamaño esperaría que fueran sus diferencias con 1000? ¿Lo observado es compatible con que las 6 sean equiprobables?

b) Los algoritmos son deterministas. Explique por qué, aun así, S_n, C_n y Q_n son variables aleatorias. Con los conteos del experimento `uniforme`, determine cuáles de los siguientes conteos varían de una lista a otra y cuáles son constantes: swaps y comparaciones de insertion sort, swaps y comparaciones de selection sort, comparaciones de quicksort. Justifique el valor de los conteos constantes. ¿Puede un conteo ser constante aunque la entrada sea aleatoria?

c) Con los conteos del experimento `desorden`, ¿qué valores toman S_{100}, C_{100} y Q_{100} sobre la lista ya ordenada ($k = 0$)? ¿Son variables aleatorias? Compare el valor de Q_{100} con los que se obtienen sobre permutaciones uniformes.

1.2. **Descripción de S_{100} y Q_{100}.** Con las $N = 2000$ listas del experimento `uniforme`:

a) Muestre las distribuciones de S_{100} y de Q_{100} con histogramas, diagramas de caja y gráficos cuantil-cuantil contra la normal. Calcule media, desvío estándar, mediana, intervalo intercuartil y coeficiente de asimetría $\hat{\gamma}$.

b) En la definición de γ_X, ¿por qué se eleva al cubo y no al cuadrado? ¿Por qué se divide por σ^3? ¿Cuánto vale γ_X si la distribución de X es simétrica alrededor de su media?

c) ¿Qué diferencias de forma observa entre S_{100} y Q_{100}? ¿Es consistente la posición relativa de la media y la mediana con el valor de $\hat{\gamma}$? ¿Alguna de las dos distribuciones se parece a una conocida?

1.3. **Un modelo para S_n.** Una *inversión* de una lista a es un par de posiciones $i < j$ con $a[i] > a[j]$. Usaremos como dato los siguientes hechos:

- S_n es igual a la cantidad de inversiones de la lista.
- Para $j = 1, \ldots, n$ sea V_j la cantidad de elementos *anteriores* a la posición j que son mayores que el elemento en la posición j, de modo que $S_n = V_1 + \cdots + V_n$. Si la lista es una permutación uniforme, las V_j son **independientes** y $V_j \sim \mathcal{U}\{0, 1, \ldots, j-1\}$.

a) Para $n = 3$, liste las 6 permutaciones de $0, 1, 2$ y, para cada una, el vector (V_1, V_2, V_3) y el valor de S_3. ¿Es consistente la tabla con el dato? Calcule $\mathrm{E}(S_3)$ y $\mathrm{Var}(S_3)$ directamente de la tabla.

b) Calcule $\mathrm{E}(V_j)$ y $\mathrm{Var}(V_j)$, y deduzca que

$$\mathrm{E}(S_n) = \frac{n(n-1)}{4}, \qquad \mathrm{Var}(S_n) = \frac{n(n-1)(2n+5)}{72}.$$

¿En qué paso usó la independencia? Verifique con el ítem a).

c) Compare μ_S y σ_S^2 con la media y la varianza muestrales del ítem 1.2. ¿De qué orden de magnitud esperaría que fuera la diferencia entre la media muestral y μ_S?

d) Simule $N = 2000$ valores de S_{100} directamente desde el modelo, sumando V_j uniformes independientes, sin correr ningún algoritmo de ordenamiento. Compare con los conteos de insertion sort mediante histogramas superpuestos y un gráfico cuantil-cuantil de una muestra contra la otra. ¿Qué esperaría ver si el modelo fuera correcto?

e) Usando la estructura de S_n, explique por qué es razonable que su distribución sea aproximadamente normal. ¿Por qué no hay un argumento análogo para Q_n?

1.4. **Comparaciones de quicksort.**

a) Compare la media y la varianza muestrales de Q_{100} con μ_Q y σ_Q^2, calculadas con las fórmulas de la sección *Resultados conocidos*.

b) Con el experimento `tamanos`, grafique en función de n el desvío relativo σ_{Q_n}/μ_{Q_n} teórico y su estimación con los datos. ¿Qué pasa cuando n crece? ¿Qué significa esto en términos de lo predecible que es el costo de quicksort para listas grandes?

Bloque 2: Simulación y estimación

En este bloque los experimentos se repiten R veces, generando listas nuevas en cada repetición. Un estimador calculado a partir de N listas es una variable aleatoria: sus R valores simulados permiten estimar su sesgo, su varianza y su error cuadrático medio, $\mathrm{ECM}(\hat{\theta}) = \mathrm{E}\left[(\hat{\theta} - \theta)^2\right] = \mathrm{sesgo}(\hat{\theta})^2 + \mathrm{Var}(\hat{\theta})$.

2.1. **Media o mediana.** Para estimar μ_Q a partir de N listas se proponen dos estimadores diferentes: la media muestral $\hat{\mu}_Q$ y la mediana muestral $\tilde{\mu}_Q$. Vamos a querer medir la calidad de dichos.

a) Para $N = 20$ y $N = 100$, estime por simulación ($R = 2000$) el sesgo, el desvío estándar y el ECM de cada estimador. ¿Cuál conviene?

b) Repita para μ_S con $\hat{\mu}_S$ y $\tilde{\mu}_S$. ¿La razón por la que un estimador resulta peor que el otro es la misma para S_{100} que para Q_{100}? ¿Qué pasa con el sesgo de $\tilde{\mu}_Q$ cuando N crece? ¿Qué parámetro estima en realidad la mediana muestral?

c) Estime γ_S y γ_Q con una simulación muy grande (por ejemplo, 10^5 listas; para S_{100} puede usar la simulación desde el modelo del ítem 1.3). Luego estudie el estimador $\hat{\gamma}$ con $N = 20, 50, 200$: estime su sesgo y su desvío para cada variable. ¿Qué tan preciso es el valor de $\hat{\gamma}_Q$ que calculó en el ítem 1.2?

2.2. **Distribución de la media muestral.** Sea $\hat{\mu}_Q$ la media de las comparaciones de quicksort sobre N listas independientes.

a) Muestre que $\mathrm{Var}(\hat{\mu}_Q) = \sigma_Q^2/N$ y que $\gamma_{\hat{\mu}_Q} = \gamma_Q/\sqrt{N}$. (Ayuda: considere $\hat{\mu}_Q - \mu_Q = \frac{1}{N}\sum_{i=1}^{N}\left(Q^{(i)} - \mu_Q\right)$ y use $D_i = Q^{(i)} - \mu_Q$, desarrolle el cubo y piense qué términos tienen esperanza nula.)

b) Verifíquelo por simulación para $N = 1, 4, 9, 25$ con $R = 2000$ valores de $\hat{\mu}_Q$: histogramas, gráficos cuantil-cuantil contra la normal, varianza y asimetría comparadas con los valores teóricos.

c) Relacione con el teorema central del límite. ¿A partir de qué N le parece razonable aproximar la distribución de $\hat{\mu}_Q$ por una normal?

2.3. **Intervalos de confianza.**

a) Construya intervalos de confianza de nivel 0,95 para μ_S y para μ_Q a partir de N listas, de dos formas: con la varianza teórica conocida (intervalo z) y con la varianza muestral (intervalo t).

b) Repita la construcción $R = 2000$ veces para $N = 5, 10, 30, 100$ y estime la cobertura, es decir, la proporción de intervalos que contienen el valor verdadero. Informe además por separado la proporción de intervalos que quedan enteramente por encima y enteramente por debajo del valor verdadero.

c) ¿La cobertura es la nominal? ¿Por qué los fallos no se reparten por igual entre los dos lados para Q_{100} y sí para S_{100}? Relacione con el ítem 2.2.

2.4. (**Optativo**) **Estimar el desorden.** Usaremos como dato que una transposición al azar aplicada a una lista ordenada de tamaño n genera en promedio $c_n = \dfrac{(n-1)(2n-1)}{3n}$ inversiones ($c_{100} \approx 65{,}67$). Se tienen N listas de tamaño $n = 100$ generadas con el mismo desorden k, que es desconocido, y se miden sus swaps de insertion sort. Llamamos $\mu_S(k) = \mathrm{E}(S_{100} \mid k)$.

a) Suponiendo que cada transposición agrega en promedio c_{100} inversiones, independientemente de las anteriores, escriba $\mu_S(k)$ en función de k y proponga un estimador $\hat{k}$ por el método de momentos.

b) Para $k = 3$ y $k = 8$, y $N = 1, 7, 15, 40$, estime por simulación ($R = 2000$) el sesgo, el desvío estándar y el ECM de $\hat{k}$.

c) ¿Cuántas listas hacen falta para que el desvío de $\hat{k}$ sea menor que 0,5? ¿Alcanza con eso para que $\hat{k}$ esté cerca de k? ¿Por qué aumentar N no corrige el sesgo? ¿Qué supuesto del ítem a) falla?

d) Estime por simulación la curva $\mu_S(k)$ para $k = 0, \ldots, 30$, grafíquela junto con la recta $c_{100}\, k$ y proponga un estimador $\tilde{k}$ que la use. Compare el ECM de $\hat{k}$ y de $\tilde{k}$. ¿Cuál conviene con $N = 1$? ¿Y con $N = 40$?

Bloque 3: Comparaciones y tests

En este bloque contrastamos hipótesis sobre los conteos. Como conocemos cómo se generan los datos, además de aplicar cada test una vez vamos a estimar por simulación con qué frecuencia rechaza: si H_0 es verdadera, esa proporción estima el **nivel** del test; si es falsa, estima su **potencia**. Todos los tests pueden calcularse con `scipy.stats`; no es necesario programar los estadísticos a mano. Como la proporción de rechazos se estima varias veces, conviene escribir una única función, por ejemplo `proporcion_rechazos(generar_par, test, R)`, que llame R veces a `generar_par()` (que devuelve un par de muestras nuevas), aplique `test` a cada par y devuelva la proporción de p-valores menores que 0,05.

Herramientas.

- **Test de Shapiro–Wilk** (`scipy.stats.shapiro`). H_0: los datos son una muestra de alguna distribución normal, con media y varianza no especificadas. No lo vimos en clase: forma parte del trabajo investigarlo.
- **Test t de Welch** (`scipy.stats.ttest_ind` con `equal_var=False`). H_0: $\mu_X = \mu_Y$, sin suponer que las dos varianzas son iguales. Usa el estadístico

$$T = \frac{\bar{X} - \bar{Y}}{\sqrt{s_X^2/N_X + s_Y^2/N_Y}},$$

  que bajo H_0 tiene distribución aproximadamente t, con grados de libertad estimados a partir de los datos.
- **Test de permutaciones** (para el ítem optativo 3.2 d). H_0: las dos muestras provienen de la misma distribución. Si H_0 es cierta, la etiqueta de cada observación (a qué muestra pertenece) no importa. Se juntan las $N_X + N_Y$ observaciones, se las reparte al azar en dos grupos de tamaños N_X y N_Y, se recalcula la diferencia de medias, y se repite esto B veces. El p-valor es la proporción de repartos en los que la diferencia, en valor absoluto, es al menos tan grande como la observada. Puede programarse a mano o con `scipy.stats.permutation_test`.
- **Intervalo bootstrap percentil** (para el ítem optativo 3.5). Para un estadístico $\hat{\theta}$ calculado a partir de una muestra: se generan B muestras bootstrap, cada una tomando N valores con reposición de la muestra original; se calcula $\hat{\theta}$ en cada una, y el intervalo de nivel 0,95 va del percentil 2,5 al percentil 97,5 de esos B valores (`scipy.stats.bootstrap` con `method="percentile"`).

3.1. Normalidad.

a) Investigue el test de Shapiro–Wilk: qué compara su estadístico y qué valores llevan a rechazar H_0. ¿Qué relación tiene con el gráfico cuantil-cuantil contra la normal?

b) Aplíquelo a S_{100} y a Q_{100} usando las primeras $N = 50$, 200, 1000 y 2000 listas del experimento `uniforme`, junto con gráficos cuantil-cuantil. ¿Cómo cambian los p-valores con N en cada caso?

c) Un único p-valor con N chico cambia mucho de una muestra a otra. Para $N = 50$, divida las 2000 listas en 40 bloques disjuntos y calcule, para S_{100} y para Q_{100}, en qué proporción de ellos el test rechaza al nivel 0,05. ¿Qué estima cada proporción? S_{100} no es exactamente normal: toma valores enteros entre 0 y 4950. ¿Por qué el test casi nunca lo detecta? ¿Se puede concluir de un no rechazo que los datos son normales? (Ayuda: recuerde la definición de potencia.)

3.2. **Nivel y potencia: el efecto del desorden.** Compare las comparaciones de quicksort Q_{100} sobre permutaciones uniformes de 100 elementos con las obtenidas sobre listas de 100 elementos con desorden k, usando $N = 50$ listas nuevas de cada tipo y $R = 300$ repeticiones.

a) **Nivel.** Compare dos muestras independientes de permutaciones uniformes con un test de Welch y estime la proporción de rechazos. ¿Qué cantidad está estimando y cuánto debería valer? (Ayuda: recuerde que el nivel es la probabilidad de rechazar H_0 cuando H_0 es verdadera, es decir, de cometer un error de tipo I; la potencia es la probabilidad de rechazar H_0 cuando H_1 es verdadera, es decir, $1 - P(\text{error de tipo II})$.)

b) **Potencia.** Para $k = 50, 75, 100, 125, 150, 200, 300$, estime la proporción de rechazos del test de Welch y grafíquela en función de k.

c) ¿A partir de qué k deja de detectarse una diferencia? Que el test no rechace, ¿significa que las dos distribuciones son iguales? Para $k = 200$, estime la proporción de rechazos con $N = 5000$ listas de cada tipo ($R = 100$ repeticiones alcanzan) y compárela con la que obtuvo con $N = 50$.

d) (**Optativo**) Repita a) y, para $k = 100$ y $k = 150$, b) con un test de permutaciones ($B = 1000$ permutaciones), usando $R = 1000$ repeticiones. Compare con el test de Welch. ¿Qué hipótesis nula contrasta cada uno? Q_{100} es asimétrica: ¿por qué, con muestras del mismo tamaño, eso casi no afecta el nivel del test de Welch? (Ayuda: con el ítem 2.2, calcule la asimetría de $\bar{X} - \bar{Y}$ cuando X e Y tienen la misma distribución y $N_X = N_Y$.)

3.3. **Comparaciones apareadas.** Como la función `medir` corre todos los algoritmos sobre las mismas listas, las mediciones de un mismo experimento pueden aparearse.

a) Con el experimento `uniforme`, calcule la correlación entre S_{100} y Q_{100}. Usando $\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) - 2\,\mathrm{Cov}(X, Y)$, ¿aparear reduciría la variabilidad de la diferencia en este caso?

b) Repita con las comparaciones C_{100} y los swaps S_{100} de insertion sort. Con $N = 10$ listas, contraste $H_0 : \mathrm{E}(C_{100}) = \mathrm{E}(S_{100})$ con un test t no apareado y con uno apareado (`scipy.stats.ttest_rel`). Repita $R = 1000$ veces con listas nuevas y compare la proporción de rechazos de ambos tests.

c) (**Optativo**) Explique la diferencia. ¿Por qué $C_{100} - S_{100}$ varía tan poco de una lista a otra? (Ayuda: en cada iteración del **para** de insertion sort, ¿cuántas comparaciones no terminan en un swap?)

3.4. **ANOVA.** Considere $g = 4$ grupos de listas con desorden $k = 100$, 150, 200 y 300, con 100 listas nuevas por grupo.

a) Plantee las hipótesis del ANOVA de un factor para comparar $\mathrm{E}(Q_{100} \mid k)$ entre los cuatro grupos, calcule el estadístico F (`scipy.stats.f_oneway`) y concluya.

b) ¿Se cumplen los supuestos del ANOVA? Compare los desvíos de los grupos y describa la forma de cada distribución.

c) Haga las 6 comparaciones de a pares con tests de Welch (que no suponen varianzas iguales) y corrección de Bonferroni. ¿Qué grupos difieren? ¿Es posible que el ANOVA rechace y ninguna comparación de a pares resulte significativa? ¿Por qué?

3.5. **Bootstrap.** La mediana de Q_{100} no tiene una fórmula sencilla. Sea δ la diferencia entre la mediana de las comparaciones de quicksort con desorden $k = 100$ y la mediana sobre permutaciones uniformes.

a) Con $N = 50$ listas de cada tipo, construya un intervalo bootstrap percentil de nivel 0,95 para δ, remuestreando cada grupo por separado ($B = 1000$).

b) Si el intervalo no contiene al 0, ¿qué se puede concluir? Repitiendo la construcción $R = 200$ veces con muestras nuevas, estime con qué frecuencia el intervalo excluye al 0. Haga lo mismo con un intervalo bootstrap para la diferencia de *medias*, y compare las dos frecuencias con la proporción de rechazos del test de Welch para $k = 100$ del ítem 3.2. ¿Por qué el intervalo basado en medianas excluye al 0 con menos frecuencia?

Forma y fecha de entrega

- Arme una notebook con la resolución del TP. No arroje código o gráficos sin explicación: intercale celdas de texto (con ecuaciones si fuera necesario), utilice Markdown para dar formato a secciones y subsecciones, etc., redacte y describa sus resultados. Las derivaciones (por ejemplo, las de los ítems 1.3 y 2.2) pueden hacerse en celdas de texto.
- Suba la notebook en el link habilitado a tal efecto (es suficiente con que lo suba un solo integrante del grupo).